до 11 янв., 01:30 | Открыть урок

Когда: 10.10.26 в 16:00

Урок 2. Как не перепутать апельсины с яблочным пирогом и посчитать его площадь. Методический материал.

- Общая информация об уроке
- Структура урока
- Актуализация: типы данных, строка из input()
- Целые числа (int) и преобразование типов
- ✅ Задача «Арифметика»
- Дробные числа (float)
- Округление: int(), round(), банковское округление
- ✅ Задача «КПД паровоза»
- Арифметические операции и приоритет
- Целочисленное деление // и остаток %
- ✅ Задача «Сотни тысяч»
- ✅ Задача «Плотность арбуза»
- Операции со строками: +, *, len(), str()
- ✅ Задача «Длина удава»
- f-строки
- min() и max()
- ✅ Задача «Буквенная замена»

- ✅ Задача «Децибелы»
- Завершение урока
- Таблица задач урока

Общая информация об уроке

Тип урока: Тематический (математика и числа в Python)
Предварительные знания учеников: переменные, `print()`, `input()`, строки в кавычках, базовые принципы PEP8

На этом уроке ученики научатся:

- Различать типы данных `int`, `float` и `str` и осознанно выбирать нужный
- Преобразовывать строки в числа (`int()`, `float()`) и числа в строки (`str()`)
- Выполнять арифметические операции, в том числе целочисленное деление `//` и остаток от деления `%`
- Использовать функцию `round()` и понимать феномен банковского округления
- Склеивать строки конкатенацией, дублировать их умножением, узнавать их длину через `len()`
- Форматировать вывод с помощью f-строк (базовый уровень)
- Применять функции `min()`, `max()`
- Подключать библиотеки через `import` и использовать базовые функции модуля `math`

Структура урока

№	Блок	Ориентировочное время	Задачи
1	Актуализация: типы данных, строка из `input()`	4 – 6 мин	—
2	Целые числа (`int`), преобразование типов	6 – 8 мин	—
3	**Задача «Арифметика»**	5 – 7 мин	Все основные операции, форматирование вывода
4	Дробные числа (`float`), экспоненциальная запись	4 – 6 мин	—
5	Округление: `int()`, `round()`, банковское округление	5 – 7 мин	—

№	Блок	Ориентировочное время	Задачи
7	Целочисленное деление `//` и остаток `%`	4 – 6 мин	—
8	**Задача «Сотни тысяч»**	5 – 7 мин	`//`, `%`, f-строки
9	**Задача «Плотность арбуза»**	5 – 7 мин	`float`, экспоненциальная запись, `math.floor`
10	Операции со строками: +, *, `len()`, `str()`	4 – 6 мин	—
11	**Задача «Длина удава»**	5 – 7 мин	`len`, `//`, `%`
12	f-строки	4 – 6 мин	—
13	`min()`, `max()`	3 – 4 мин	—
14	**Задача «Буквенная замена»**	5 – 7 мин	`min`, `max`, `len`, `//`, *
15	Модуль `math`: `import`, базовые функции	5 – 7 мин	—
16	**Задача «Третий закон Кеплера»**	5 – 7 мин	`math`, дробные числа, `round`
17	**Задача «Децибелы»**	5 – 7 мин	`math.log10`, `math.ceil`
18	Подведение итогов	2–3 мин	—

⏱ **Урок насыщен материалом** — темп сильно зависит от группы. Задачи «Буквенная замена» и «Децибелы» можно перевести в домашние или сделать «буфером» для быстрых учеников.

Актуализация: что мы уже знаем

Время: 4 – 6 минут.

Ключевые идеи для повторения

- `input()` **всегда** возвращает строку. Это важно зафиксировать в самом начале урока, потому что весь блок про типы данных вырастает именно из этого ограничения.

Рекомендация: Покажите фрагмент кода и спросите у учеников, что произойдёт:

```
a = input()       # Пользователь вводит '5'
print(a + 1)      # ???
```

Можно дать ученикам самим попробовать запустить — пусть увидят ошибку (`TypeError`). Задайте вопрос: «Почему так получилось?» и выйдите на объяснение типов данных.

Целые числа (`int`) и преобразование типов

Время: 6 – 8 минут.

Ключевые идеи для объяснения

- Для целых чисел в Python используется тип **`int`**. Кавычки не нужны.
- Ограничений по размеру нет — Python работает с сколь угодно большими целыми.
- Для читаемости разрядов разрешено использовать _ внутри числа: `999_999_999`.
- Чтобы превратить строку из `input()` в число, используется функция `int()`.

```
watermellons_count = 200
big_number = 12345678901234567890123456789O
desired_salary = 999_999_999_999

# Конвертация строки в число
five_as_string = '5'
five_as_int = int(five_as_string)

stars_count = int(input())
```

Рекомендация: Объясните вложенные вызовы функций на примере `int(input())`. Разберите пошагово: сначала выполняется `input()` и возвращает строку, затем строка передаётся в `int()`. Аналогия: матрёшка — результат одной функции «кладётся» внутрь другой.
Покажите, что произойдёт, если передать в `int()` нечисловую строку (`int('abc')`) — возникнет `ValueError`. Важно не пугаться ошибок, а читать их сообщения.

✅ Задача «Арифметика»

Классная **Что проверяется:** все базовые операции (+, -, *, **), точное форматирование строки вывода.

> ⚠️ **Частая ошибка:** ученики пишут `2^3` вместо `2 ** 3` или забывают, что в выводе нужен именно **, а не результат. Стоит разобрать это явно.

Как вести на уроке: акцентируйте: ровно по одному пробелу вокруг операторов в выводе — тестирующая система проверяет посимвольно. Обсудите: как вывести `2 ** 3 = 8` — здесь ** нужно вывести как текст, а не вычислять.

Дробные числа (`float`)

Время: 4 – 6 минут.

- Для дробных чисел используется тип **`float`**. Разделитель целой и дробной части — **точка**, не запятая.
- Поддерживается экспоненциальная запись: `6e23` — это 6 * 10^23.
- Преобразование строки в дробное число: `float()`.

```
price = 5.0
temperature = -15.5
pi = 3.14

avogadro = 6e23         # 600000000000000000000000
electron_mass = 9.1e-31

float_from_input = float(input())   # Разделитель — точка!
```

> 📌 **Спецзначения:** `float('+infinity')` → `inf`, `float('-infinity')` → `-inf`, `float('nan')` → `NaN`. Можно упомянуть вскользь.

Рекомендация: обязательно акцентируйте: **запятая вместо точки вызовет `ValueError`**. Это частая ошибка при первом знакомстве с `float`.
Экспоненциальная запись — хороший повод для метапредметного момента: «Где

Округление: `int()`, `round()`, банковское округление

Время: 5 – 7 минут.

- `int(3.9)` → 3: дробная часть просто отбрасывается (не округляется).
- `round(x)` — округление к ближайшему целому.
- Пограничный случай: `round(2.5)` → 2, `round(3.5)` → 4 — **банковское округление** (к ближайшему чётному).
- `round(x, n)` — второй параметр указывает количество знаков после запятой.

```
price = 999.99
price_in_my_head = int(price)     # 999

print(round(3.6))   # 4
print(round(3.5))   # 4
print(round(2.6))   # 3
print(round(2.5))   # 2 — банковское округление!

print(round(3.14, 1))    # 3.1
print(round(999, -3))    # 1000
```

Рекомендация: банковское округление — отличная точка удивления. Запустите в классе вместе, объясните причину: такое округление уменьшает систематическую погрешность. Практическое правило: «Не полагайтесь на `round()` при `.5` — уточняйте условие задачи».

✅ Задача «КПД паровоза»

Классная **Что проверяется:** `float(input())`, деление `/`, умножение на 100, `round()`.

⚠️ **Типичная ошибка:** забыть умножить на 100 или напутать с делением (поделить A на 100, а не Q).

Как вести на уроке: запишите формулу на доске. Попросите учеников написать код поэтапно: сначала считать ввод, затем формулу, затем вывести результат. Обратите внимание: если использовать `int(input())`, получите ошибку на дробном вводе.

Время: 5 – 7 минут.

- +, -, *, / (всегда `float`), **, `abs()`
- Если хотя бы один операнд — `float`, результат `float`.

```
print(10 + 3)    # 13
print(10 / 3)    # 3.333...
print(10 ** 3)   # 1000
print(4 ** 0.5)  # 2.0
print(abs(-5))   # 5
```

Приоритет (как в математике): скобки, степень, `* / // %`, `+ -`.

Покажите живьём: одно и то же выражение даёт разный результат со скобками и без. Попросите учеников предсказать результат до запуска.

Целочисленное деление `//` и остаток `%`

Время: 4 – 6 минут.

- `//` — сколько раз одно число **полностью** помещается в другое.
- `%` — остаток от деления.
- `divmod(a, b)` — возвращает пару.

```
pirates = 10
gold_coins = 103

coins_per_pirate = gold_coins // pirates   # 10
leftover_coins = gold_coins % pirates      # 3

# То же через divmod
coins_per_pirate, leftover_coins = divmod(gold_coins, pirates)
```

Метафора: `//` — сколько целых кусков пиццы достанется каждому, `%` — сколько останется. Или «делим золото среди пиратов». Самая частая практическая задача:

✅ Задача «Сотни тысяч»

Классная **Что проверяется:** `//` и `%` для извлечения цифр числа; форматированный вывод.

```
n = int(input())
k = (n // 100_000) % 10
print(f'В числе {n} сотен тысяч ровно {k}.')
```

Обратите внимание на точку в конце строки — её часто забывают, а тестирующая система проверяет посимвольно.

✅ Задача «Плотность арбуза»

Классная / Домашняя

Что проверяется: `float`, единицы измерения (см³ → м³ через `1e-6`), округление вниз (`int()`).

Как вести: запишите формулу. Обсудите: почему нужно именно округление вниз, а не round? В физике не принято завышать результат.

Операции со строками: +, *, `len()`, `str()`

Время: 4 – 6 минут.

- **Конкатенация** +: соединяет строки.
- **Умножение** *: дублирует строку.
- **`len()`**: возвращает количество символов.
- **`str()`**: преобразует число в строку.

```
boring_days = 'пн, вт, ср, чт, пт'
good_days = 'сб, вс'
week = boring_days + ', ' + good_days

laugh = 'ха' * 3          # 'хахаха'
```



```
text = 'Строка'
```

```
print(len(text))           # 6
print(len(''))             # 0

num1 = 3
num2 = 10
my_str = str(num1 * num2) * num1   # '303030'
```

Упомяните термин **полиморфизм**: один оператор + работает по-разному для чисел и строк. Типичная ошибка: сложение строки и числа без `str()`.

✅ Задача «Длина удава»

Классная **Что проверяется:** `len()` на строке, `//` и `%`.

Важно: ввод — две **строки**, не числа. `int()` не нужен. Задача сводится к `len(boa) // len(parrot)` и `len(boa) % len(parrot)`.

Формат вывода: два числа через пробел.

f-строки

Время: 4 – 6 минут.

```
name = 'Вася'
age = 15
print(f'Меня зовут {name}, и мне {age} лет.')

num1, num2 = 2, 2
print(f'{num1} + {num2} = {num1 + num2}')

pi = 3.1415926535
hours, minutes = 2, 9
print(f'π ≈ {pi:.2f}')              # π ≈ 3.14
print(f'{hours:02d}:{minutes:02d}') # 02:09
```

Совет: запоминать весь синтаксис не нужно — достаточно знать `:.2f` и базовую

`min()` и `max()`

Время: 3 – 4 минут.

- Находят минимум и максимум среди аргументов.
- Работают с числами и строками (лексикографически).

```
print(max(1, 2.8, 3, 4.1, 5))    # 5
print(min(-5, 10, 3.2))           # -5
print(min('Иван', 'Анна', 'Ян'))  # Анна
max_letter = max('Одна строка')   # 'т'
```

Нельзя смешивать числа и строки в одном вызове `min`/`max` — будет ошибка.

✅ Задача «Буквенная замена»

Классная **Что проверяется:** `min()` и `max()` на строке, `len()`, `//`, умножение строки.

Логика: `min_char = min(s)`, `max_char = max(s)`, `half = len(s) // 2`, затем `min_char * half + max_char * half`. Нечётная длина обрабатывается автоматически через `//`.

Модуль `math`

Время: 5 – 7 минут.

- **Модуль (библиотека)** — готовый набор функций.
- Импорт в начале файла: `import math`.
- Обращение через точку: `math.имя_функции()`.

```
import math

print(math.pi)
print(math.sqrt(25))   # 5.0
print(math.ceil(2.3))  # 3
print(math.floor(2.9)) # 2
```

```
print(math.log(100, 10))  # 2.0
print(math.log10(100))    # 2.0
```

PEP 8: `import` пишется в самом начале файла и отделяется от основного кода пустой строкой. Объясните идею модульности: никто не держит все инструменты на столе — берёт чемоданчик, когда нужно.

✅ Задача «Третий закон Кеплера»

Классная **Что проверяется:** формула с дробной степенью, `float(input())`, `round(x, 3)`.

```
import math
T = float(input())
a = T ** (2 / 3)
print(round(a, 3))
```

Некоторые ученики попробуют использовать
`math.sqrt`
дважды — стоит показать элегантность
`** (2 / 3)`
.

✅ Задача «Децибелы»

Классная / Домашняя **Что проверяется:** `math.log10`, `math.ceil`, экспоненциальная запись `1e-12`.

```
import math
I0 = 1e-12
I = float(input())
beta = 10 * math.log10(I / I0)
print(math.ceil(beta))
```

Округление **вверх** — `math.ceil`, не round

Завершение урока

Время: 2–3 минуты.

- **Домашние задачи:** перечислите, какие задачи остались невыполненными в классе.
- **Где задавать вопросы:** чат задачи в ЛМС и/или групповой чат.
- **Домашнее задание:** перечитать урок в учебнике, выполнить домашние задачи.

Таблица задач урока

Классная работа

Задача	Баллы	Что проверяется	На что обратить особое внимание
Арифметика	10	Основные арифметические операции; точное форматирование строк вывода	Операнды и операторы разделены ровно одним пробелом. В выводе ** — это текст, а не результат
КПД паровоза	10	`float(input())`, деление `/`, умножение на 100, `round()`	Использовать `float`, а не `int`. Обсудить, выполняется умножение на 100 до или после округления
Сотни тысяч	10	`//` и `%` для выделения цифры числа; f-строки	Точка в конце строки вывода. Правильная цепочка: сначала `// 100_000`, затем `% 10`
Плотность арбуза	15	`float`; экспоненциальная запись `1e-6`; округление вниз	Для округления вниз использовать `math.floor()` или `int()`. Не забыть перевести см³ в м³
Длина удава	10	`len()` на строке; `//` и `%`	Вводятся две строки, а не числа: здесь не нужен `int(input())`
Буквенная замена	15	`min()`, `max()` на строке; `len()`, `//`, умножение	Нечётная длина обрабатывается через `//`. Заглавные буквы при

Задача	Баллы	Что проверяется	На что обратить особое внимание
Третий закон Кеплера	15	Дробная степень `** (2 / 3)`; `float(input())`; `round(x, 3)`	Показатель степени записывается в скобках `(2 / 3)`, иначе приоритет операций изменит результат
Децибелы	15	`math.log10()`, `math.ceil()`; экспоненциальная запись `1e-12`	Округление вверх выполняется через `math.ceil()`, а не `round()`. `I0` задаётся константой в коде

Домашние задачи

Задача	Баллы	Что проверяется	На что обратить особое внимание
Парная сумма	15	Целочисленное деление `// 100`; остаток `% 100`; сумма двух частей числа	Первая операция выделяет первые цифры, вторая — последние две. Обычное деление `/` здесь даст дробное число
Барометр	15	`int(input())`; разность двух измерений; модуль числа через `abs()`	Порядок вычитания не влияет на ответ только благодаря `abs()`. Без него результат может оказаться отрицательным
Повторите, пожалуйста!	25	Скобки и приоритет арифметических операций; `str()`; конкатенация и повторение строки	Сначала вычислить сумму и произведение чисел, затем превратить результат в строку. Умножение до `str()` даст совсем другой результат
Жара и мороз	25	Пять значений `float`; функции `min()` и `max()`; подстановка во f-строки	Все пять значений должны участвовать в обеих функциях. Текст, регистр и точки в конце строк должны совпадать с условием
Спиральные		Строковый и числовой ввод: `len()`; умножение	`sleeves` нужно преобразовать в `int`, а `stars` оставить строкой.



`len(stars)`